42. Kubernetes-Oriented Go: Controllers and client-go

42. Kubernetes-Oriented Go: Controllers and client-go

Phase: Phase 6 — Systems and infrastructure
Estimated time: 2–3 days
Slug: `42-kubernetes-go`

Learning objectives

- Explain control loops (reconcile)
- Sketch a controller with client-go concepts
- Use leader election awareness
- Design for level-triggered reconciliation

Prerequisites

Complete prior numbered lessons. Type every example yourself.

Control loop

Kubernetes controllers observe desired state (spec) and drive actual state toward it — continuously.

```
for {
  desired := getSpec()
  actual := getStatus()
  if actual != desired { act() }
}
```

client-go concepts

Informers/caches reduce API spam; workqueues debounce; reconcile must be idempotent.

Operators

Encode domain ops knowledge as controllers. Go dominates this ecosystem.

You do not need a cluster for the exercise — implement a tiny reconcile function over an in-memory store.

Common mistakes (especially from Python instincts)

- One-shot scripts instead of reconcile loops
- Non-idempotent reconciles
- Hot-looping without backoff

Exercises

Work in `exercises/42-kubernetes-go/`. Read that folder's `README.md`.

- **A (Drill):** make unit tests pass with the simplest correct code.

- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/42-kubernetes-go/` until you have an honest failing attempt.

Self-check

1. What is reconcile?
2. Why idempotent?
3. Role of informers?

Further reading

- https://github.com/kubernetes/client-go